

ASSIGNMENT3 SDP

SE-2527 ANEL MELIS

Topic B-notifications

Repository: <https://github.com/anelmelis/asik3sdp>

Base Commit: 91709fdf03136419899972ff2305486fbe6adcb2

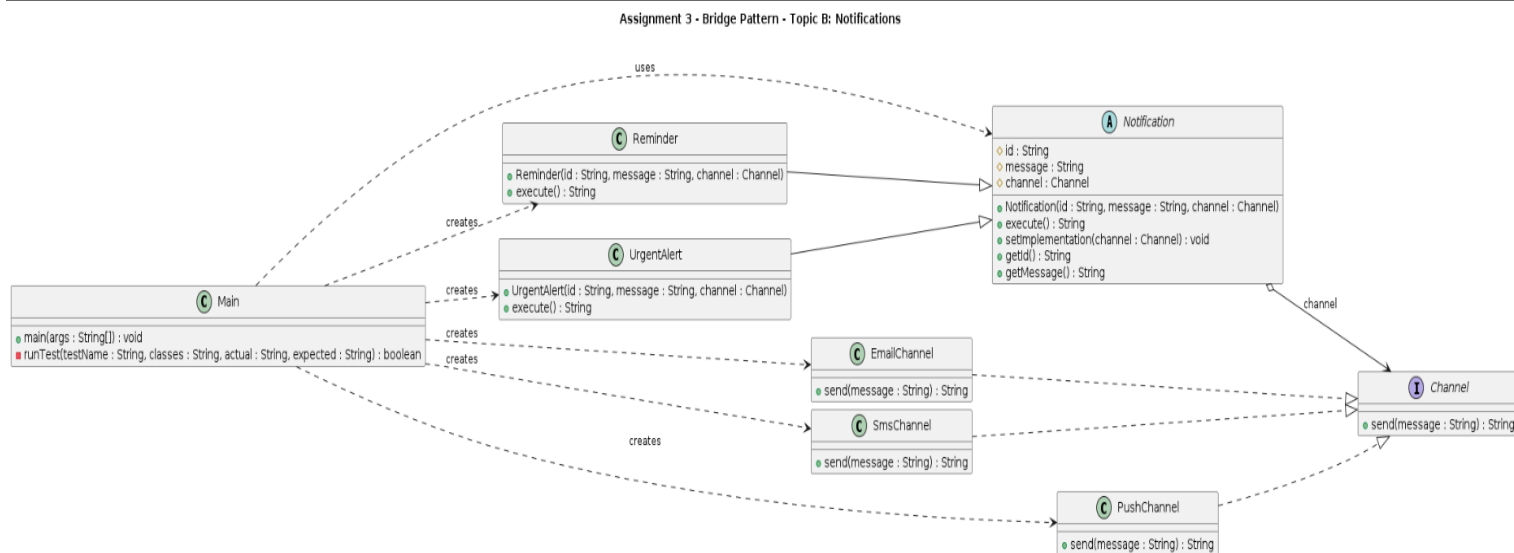
1. Bridge Pattern

This project uses the Bridge Design Pattern for a notification system. There are two independent dimensions:

- notification types: Notification, Reminder, UrgentAlert;
- delivery channels: Channel, EmailChannel, SmsChannel, PushChannel.

Bridge fits because notification types and delivery channels can change independently without creating a separate class for every combination.

2. UML Class Diagram



The Bridge link is the field: protected Channel channel;

Notification depends on the Channel interface, not on concrete channel classes. Reminder and UrgentAlert extend Notification. EmailChannel, SmsChannel, and PushChannel implement Channel.

3. Clean Code Examples

Separation of responsibilities

Notification classes define notification behavior, while Channel classes define delivery behavior.

```
public interface Channel {  
    String send(String message);  
}
```

Meaningful names

Names such as Reminder, UrgentAlert, EmailChannel, and SmsChannel clearly describe the purpose of each class.

Small focused methods

```
public String execute() {  
    return channel.send("Reminder: " + message);  
}
```

The method has one clear purpose and delegates delivery to the channel.

Avoiding duplication

The project does not create classes such as EmailReminder or SmsReminder.

The same Reminder can work with different channels.

Encapsulation of state

Notification stores its own data:

```
protected String id;  
protected String message;  
protected Channel channel;
```

Only the channel reference changes during runtime switching.

4. Runtime Switch

In T5, the same Reminder object first uses EmailChannel and then changes to SmsChannel.

```
switchTest.setImplementation(new SmsChannel());
```

Object identity is checked using:

```
originalReference == switchTest
```

The result is true, while the ID and message remain unchanged.

This proves that only the implementation changes, not the abstraction object.

5. Extension with PushChannel

The base version contained `EmailChannel` and `SmsChannel`.

After the base commit, `PushChannel` was added:

```
public class PushChannel implements Channel {  
    public String send(String message) {  
        return "PUSH: " + message;  
    }  
}
```

The existing `Notification`, `Reminder`, `UrgentAlert`, `Channel`, `EmailChannel`, and `SmsChannel` classes were not changed.

This demonstrates independent extension of the implementation hierarchy.

6. Trade-off

A disadvantage of Bridge is that it introduces more classes and interfaces.

However, it becomes useful when both sides of the system may grow independently.

7. Bridge vs Adapter

Bridge separates abstraction and implementation so they can evolve independently.

Adapter is used to make incompatible existing interfaces work together.

In this project, Bridge separates notification type from delivery channel.

8. Result

All required checks pass:

SUMMARY: 7/7 PASS

9. References

1. Course Lecture 4 – Bridge Pattern.
2. Moodle Bridge Pattern materials.
3. Assignment 3 – Bridge Pattern instructions.
4. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*.